ChatGPT

Executive Summary

Preparing for an Amazon SDE (Software Development Engineer) intern interview requires understanding the multi-stage process and practicing both coding and behavioral skills. Amazon's intern hiring typically begins with a recruiter screen and online assessment (coding test + work-style survey) [1]. Successful candidates are then invited to interview rounds (often 45–60 minutes each) combining coding problems and behavioral/Leadership Principle (LP) questions [2] [3]. Coding questions frequently cover common data structures (arrays, strings, lists, trees, graphs, etc.) and standard problem types (two-pointer, sliding window, DFS/BFS, DP, bit tricks, etc.) [4] [2]. Behavioral rounds focus on Amazon's Leadership Principles (e.g., Ownership, Customer Obsession, Bias for Action) [5] [6]. This report details the interview structure and stages, lists representative recent technical questions by topic (with explanations, complexities, and sample Python/Java solutions), summarizes typical behavioral questions with answer frameworks, offers on-campus interview tips, and proposes a 2–4 week preparation plan. It also includes tables showing the relative frequency of topics and recommended practice problems (with links). **Sources:** Amazon's careers site and recent interview reports [1] [2] [3] [4] and candidate experiences [7] [8] [9].

```
flowchart LR
    A[Application & Recruiter Screen] --> B[Online Assessment\n(Coding +
Workstyles) [1]]
    B --> C{Technical Interviews}
    C -->|Round 1| D[Coding & Behavioral (45–60m) [2]]
    C -->|Round 2 (if any)| E[Coding & Behavioral (45–60m)]
    D --> F[Decision: Offer / No Offer]
    E --> F
```

Figure: Typical Amazon SDE Intern interview flow. Initial screening and the OA (coding + workstyle) precede one or two interview rounds (each mixing coding and LP questions) [1] [2].

Interview Structure and Stages

Amazon's intern process **begins** with a recruiter or resume screen. Next is an **Online Assessment (OA)** of ~90 minutes (70 min coding + ~15 min work-style survey) [1]. The coding portion typically has 2–3 problems (easy–medium level), often one question per problem [10] [2]. For example, one intern reported two string-manipulation problems (easy-medium) in the OA [2]. After passing the OA, candidates are invited to **interview rounds**. At campuses or virtually, this often means one or two 45–60 minute sessions with engineers. In each interview, you'll usually start with brief introductions, answer 1–2 behavioral (LP) questions, then solve one or two coding problems [11] [12]. Some interviews may include a low-level design or coding puzzle question (e.g. a "bus fare calculator") [13], but core focus remains data structures and algorithms. Most coding questions are LeetCode-style (arrays, strings, trees, graphs, DP, bit operations, etc.) [4]. After interviews, the team debriefs and a decision is made (offer or not) [14].

Technical Question Examples (by topic)

Arrays

- **Two Sum** (LeetCode "Two Sum"). *Difficulty: Easy.* Given an integer array and a target, find indices of two numbers summing to the target. Use a hash map to store seen values and check complements in O(n) time and O(n) space.
  Time: O(n), *Space:* O(n).
  Python:

```python
def twoSum(nums, target):
    d = {}
    for i, num in enumerate(nums):
        if target - num in d:
            return [d[target-num], i]
        d[num] = i
    return []
```

Java:

```java
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer,Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int comp = target - nums[i];
            if (map.containsKey(comp)) {
                return new int[]{map.get(comp), i};
            }
            map.put(nums[i], i);
        }
        return new int[]{};
    }
}
```

- **Best Time to Buy and Sell Stock** (LeetCode "Best Time to Buy and Sell Stock"). *Difficulty: Easy.* Given array of prices, find max profit from one transaction. Scan once, tracking the minimum price seen so far and updating max profit.
Time: O(n), *Space:* O(1).
Python:

```python
def maxProfit(prices):
    min_price = float('inf')
    profit = 0
    for p in prices:
```

```
        min_price = min(min_price, p)
        profit = max(profit, p - min_price)
    return profit
```

Java:

```
class Solution {
    public int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE, profit = 0;
        for (int p : prices) {
            minPrice = Math.min(minPrice, p);
            profit = Math.max(profit, p - minPrice);
        }
        return profit;
    }
}
```

- **Count of Buttons Pressed in Keypad** (variation). One interviewee reported a variant: *"Count of buttons pressed in a keypad mobile"* [15] (a dynamic programming on numeric keypad problem).
- **Sort Array by Custom Order** (Strings/arrays). Another OA problem was *"Sort an array of strings based on a given order"* [15]. Typically involves a custom comparator based on a mapping.

Strings

- **Longest Substring Without Repeating Characters** (LeetCode 3). *Difficulty: Medium.* Given a string, find the length of the longest substring without repeating characters. Use the sliding-window technique: maintain a window with no duplicates, moving two pointers. Store the last index of each character in a hash map (or fixed array) to update the left pointer in O(1) when a repeat is found [16] [17].
  Time: O(n), *Space:* O(min(n, alphabet_size)).
  Python:

```
def longestUnique(s):
    last = {}
    start = 0
    res = 0
    for i, ch in enumerate(s):
        if ch in last and last[ch] >= start:
            start = last[ch] + 1
        res = max(res, i - start + 1)
        last[ch] = i
    return res
```

Java:

```java
class Solution {
    public int lengthOfLongestSubstring(String s) {
        Map<Character,Integer> last = new HashMap<>();
        int start = 0, res = 0;
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (last.containsKey(c) && last.get(c) >= start) {
                start = last.get(c) + 1;
            }
            res = Math.max(res, i - start + 1);
            last.put(c, i);
        }
        return res;
    }
}
```

Remarks: An interviewer may push for further optimization. For example, by using a fixed-size array (size 256 or 128) instead of a hash map, one can achieve *O(1) space (for ASCII)*, as the candidate did here [18].

- **Recursively Remove Adjacent Duplicates** (GeeksforGeeks). *Difficulty: Medium.* Given a string, repeatedly remove pairs of adjacent duplicate characters until no more remain [19]. Example: “azxxzy” → “azzy” → “ay”. One can solve by recursion or stack. A recursive strategy (from GfG) removes one adjacent run at a time:
  Python:

```python
def removeAdjDupRec(s):
    if len(s) < 2: return s
    i = 1
    while i < len(s):
        if s[i] == s[i-1]:
            # skip all identical chars s[i-1..j-1]
            j = i+1
            while j < len(s) and s[j] == s[i]:
                j += 1
            # remove the run and recurse
            return removeAdjDupRec(s[:i-1] + s[j:])
        else:
            i += 1
    return s
```

Java:

```java
class Solution {
    public String removeAdjacentDuplicates(String s) {
        if (s == null || s.length() < 2) return s;
```

```
        int i = 1;
        while (i < s.length()) {
            if (s.charAt(i) == s.charAt(i-1)) {
                int j = i+1;
                while (j < s.length() && s.charAt(j) == s.charAt(i)) j++;
                String reduced = s.substring(0, i-1) + s.substring(j);
                return removeAdjacentDuplicates(reduced);
            } else {
                i++;
            }
        }
        return s;
    }
}
```

Complexities: Each removal shortens the string; worst-case time can be $O(n^2)$ due to repeated recursion, but typically O(n).

- **Reorganize String** (LeetCode 767). *Difficulty: Medium.* Re-arrange characters so no two identical are adjacent. Use a max-heap (priority queue) of character counts to always pick the two most frequent remaining chars and append them. If impossible, return empty.
  Approach: Greedy with a max-heap.
  Time: O(n log k), *Space:* O(k) where k = #distinct chars.

Linked Lists

(Examples:)

- **Reverse Linked List.** *Difficulty: Easy.* Iteratively or recursively reverse `ListNode` pointers. *Time:* O(n), *Space:* O(1) (iterative).
- **Detect Cycle in Linked List.** *Difficulty: Easy.* Use Floyd's cycle-finding (fast/slow pointers) to detect a loop. *Time:* O(n), *Space:* O(1).
- **Merge Two Sorted Lists.** *Difficulty: Easy.* Classic: merge like in merge sort. *Time:* O(n), *Space:* O(1).

Trees & Tries

(Examples:)

- **Invert/Flip Binary Tree.** *Difficulty: Easy.* Recursively swap left/right children of each node. *Time:* O(n), *Space:* O(h) recursion.
- **Binary Tree BFS (Level Order).** *Difficulty: Easy/Medium.* Use a queue to traverse by level. *Time:* O(n).
- **Serialize/Deserialize Binary Tree.** *Difficulty: Hard.* Example LLD question for serialization.
- **Trie-based problems:** Building or searching a Trie for string collections.

Graphs

- **Course Schedule** (LeetCode 207). *Difficulty: Medium.* Detect if a directed graph has a cycle (prerequisite list) [20] . Approach: Topological sort or DFS cycle detection.

Example Wording: "Given `numCourses` and a list of prerequisite pairs [course, prereq], determine if you can finish all courses." *Time:* O(V+E), *Space:* O(V+E).
Python:

```python
def canFinish(numCourses, prerequisites):
    graph = {i: [] for i in range(numCourses)}
    for a,b in prerequisites:
        graph[b].append(a)
    visit = [0]*numCourses  # 0=unvisited,1=visiting,2=done
    def dfs(i):
        if visit[i] == 1: return False
        if visit[i] == 2: return True
        visit[i] = 1
        for nei in graph[i]:
            if not dfs(nei): return False
        visit[i] = 2
        return True
    for i in range(numCourses):
        if not dfs(i): return False
    return True
```

Java: (Similar DFS approach.)

- **Pacific Atlantic Water Flow** (LeetCode 417). *Difficulty: Medium.* Find all grid cells from which water can flow to both the Pacific and Atlantic edges [8]. Approach: Do two multi-source BFS/DFS from the Pacific and Atlantic borders. Each BFS marks reachable cells; take the intersection. *Time:* O(m*n),* Space: *O(mn*).
  Python:

```python
from collections import deque
def pacificAtlantic(heights):
    if not heights: return []
    m, n = len(heights), len(heights[0])
    pac = [[False]*n for _ in range(m)]
    atl = [[False]*n for _ in range(m)]
    dirs = [(1,0),(-1,0),(0,1),(0,-1)]
    def bfs(starts, ocean):
        q = deque(starts)
        while q:
            x,y = q.popleft()
            for dx,dy in dirs:
                nx,ny = x+dx, y+dy
                if (0<=nx<m and 0<=ny<n and not ocean[nx][ny]
                    and heights[nx][ny] >= heights[x][y]):
                    ocean[nx][ny] = True
                    q.append((nx, ny))
```

```
    pac_starts = [(i,0) for i in range(m)] + [(0,j) for j in range(n)]
    atl_starts = [(i,n-1) for i in range(m)] + [(m-1,j) for j in range(n)]
    for x,y in pac_starts: pac[x][y] = True
    for x,y in atl_starts: atl[x][y] = True
    bfs(pac_starts, pac); bfs(atl_starts, atl)
    return [[i,j] for i in range(m) for j in range(n) if pac[i][j] and atl[i]
[j]]
```

Java: (Use two BFS with queues from both borders.)

- **"Team Dominance Order"** – Essentially a cycle-detection in directed graph (same as Course Schedule). One intern phrased: "Given pairs [A,B] meaning A dominates B, check if a valid order exists" [21] . This is topological sort. The candidate used Kahn's algorithm.

Dynamic Programming

- **0/1 Knapsack**, **Coin Change**, **Longest Common Subsequence**, etc. *Difficulty:* Medium. Standard DP problems. Amazon often expects you to recognize overlapping subproblems and optimize.
- **Partition Equal Subset Sum**, **Jump Game**, **Word Break**, etc. Use boolean DP or backtracking+memo.

Bit Manipulation & Math

- **Count Set Bits**, **Power of Two**, **XOR trick (Single Number)**, etc. *Difficulty:* Easy/Medium. For example, "Single Number" uses XOR in O(n).
- **Prime checking**, **GCD**, **Exponentiation by squaring** might come up in math puzzles.

Coding Puzzles & Others

Amazon may include one "puzzle" or tricky problem in an interview to test reasoning. Examples (less common) include classic puzzles: **Water Jug Problem**, **N-Queens**, **Word Ladder**, **Sudoku Solver** (backtracking), etc. While not guaranteed, candidates are sometimes asked to think aloud on such problems. Amazon also asks simple complexity analysis questions (e.g. *"What is the time complexity of your solution?"*), so always state and, if possible, justify Big-O during your solution.

Behavioral / Leadership-Principle Questions

Amazon heavily emphasizes its 16 **Leadership Principles** in interviews [5] [6] . Expect **behavioral** questions framed as "Tell me about a time when…", mapped to LPs. For each, use the **STAR** framework (Situation, Task, Action, Result) and **explicitly name the LP** you're illustrating. For example:

- **Ownership**: *Question:* "Describe a time when you took ownership of a project."
  Framework: Highlight that you "acted as an owner, thinking long-term and driving results" [6] .
  Sample Answer: "In my last internship, I inherited a stalled feature outside my expertise. I **owned** it by quickly learning the new framework, setting milestones, and communicating progress daily. I negotiated scope with my mentor and, despite tight deadlines, delivered a robust solution on time.

This demonstrated Ownership (taking responsibility beyond my role) and Earn Trust (by updating stakeholders) [6] [22] ."

- **Customer Obsession/Bias for Action**: *Question:* "Give an example of improving a product or process based on user feedback."
  Framework: Show how you prioritized customers and acted decisively [5] [23] .
  Sample Answer: "In a class project, I noticed students struggled with a lab tool. I gathered feedback, then quickly prototyped a simpler UI (Bias for Action). The next week's survey showed a 50% drop in confusion. I talked about Customer Obsession ("I started from their needs") and Bias for Action (rapid small improvements) [5] [23] ."

- **Disagree and Commit**: *Question:* "Describe a time you disagreed with a team decision."
  Framework: Focus on respectful challenge and commitment once a decision is made [24] .
  Sample Answer: "During a hackathon, my team wanted to build X, but I believed feature Y would add more value (Are Right, A Lot). I presented data politely and convinced them to pivot (Earn Trust, Have Backbone) [25] [24] . When final decisions were made, I fully supported the team direction and helped implement it (Disagree and Commit)."

(For other LPs like Learn and Be Curious, Dive Deep*, etc., prepare similar STAR stories. One candidate noted that relating answers to LPs was appreciated by interviewers [26] .)*

On-Campus Interview Tips

- **Clarify & Ask Questions:** Always clarify constraints and examples before coding. Amazon interviewers often expect you to **ask clarifying questions and discuss edge cases** [26] . For example, one intern was prompted mid-answer to refine scope, which is normal.
- **Think Aloud:** Explain your thought process step-by-step. Amazon values seeing your reasoning. Walk through naive vs. optimized ideas. ("I first consider X, then realize Y..."). In one experience, the candidate first explained a brute force ($O(n^2)$) then optimized to $O(n)$ [27] .
- **Coding Environment:** On-campus interviews traditionally use a whiteboard or paper, but many are now virtual (shared coding editor or IDE). Write code neatly, define data structures, and use meaningful variable names. If doing whiteboard, draw any structures or states.
- **Time Management:** Interview rounds are tight (usually 45–60 minutes). For a single question, spend up to 5 minutes outlining your approach before coding. If stuck, communicate partial thoughts and ask the interviewer for hints or next steps.
- **Language Choice:** Use whichever language you're most fluent in (Python, Java, C++, etc.). Make sure you know syntax for basic operations in that language.
- **Follow-ups:** Be ready to discuss time/space complexity (e.g., one intern was explicitly asked to optimize and state complexities [28] [29]). If performance can be improved, mention possible trade-offs.
- **Behavioral Cues:** Interviewers may interrupt to dig deeper. Answer succinctly and bring back to the STAR points if diverted.
- **Impression:** Be professional but personable. One candidate noted a friendly vibe helped: "Interviewer was chill... made the process enjoyable" [30] . Treat the interview as a two-way conversation; having thoughtful questions ready is positive.

Sample 2–4 Week Study Plan

Week 1: Core DSA Review. Refresh fundamentals and easy problems.

- **Mon:** Revise language/library (array ops, maps, etc.). Solve 2–3 array problems (e.g. Two Sum, Move Zeroes).
- **Tue:** Strings – solve problems like Longest Substring Without Repeat, Valid Palindrome, Reorganize String.
- **Wed:** Linked Lists – reverse list, detect cycle, merge lists.
- **Thu:** Trees – BFS/DFS traversal, invert tree, LCA.
- **Fri:** Graph basics – adjacency list, BFS/DFS practice, Course Schedule (cycle detection).
- **Sat:** Dynamic Programming basics – 1D DP (Climbing Stairs, Best Time to Sell).
- **Sun:** Mock interview: pick 2 problems (e.g. one array, one tree) and simulate explaining and coding on a whiteboard or shared doc.

Week 2: Intermediate Challenges & Systems. Tackle medium-level problems and system knowledge.

- **Mon:** Advanced arrays/strings – subarray sums (Kadane, prefix sums), sliding window (Minimum Window Substring).
- **Tue:** Trees & Heaps – Kth largest in array (heap), median stream, or Next Permutation (array).
- **Wed:** Graphs – Pacific Atlantic Water Flow, Number of Islands, Topological sort.
- **Thu:** DP – 2D DP (Knapsack, Word Break), recursion/backtracking (N-Queens, Sudoku).
- **Fri:** Bit manipulation & Math – Single Number (XOR), count bits, power of two.
- **Sat:** System/LLD basics – quick read on designing small systems or class designs (e.g. “design an in-memory cache”), though full design usually more for SDE1 than intern.
- **Sun:** Practice behavioral questions (write STAR answers for 5–6 LPs); do one mock interview with combined coding + behavioral.

Week 3: Advanced Prep & Mock Interviews.

- **Mon:** Solve 2–3 new medium problems (LeetCode “Amazon” tag) in a timed setting; review common algorithms (sorting, binary search).
- **Tue:** Focus on weakness: if DP is weak, do more DP; if graphs, do more graph.
- **Wed:** Do a full 45-minute mock on **interview platforms** (CoderPad, Pramp) or with a peer/mentor. Use a timer. Simulate an Amazon interview (2-3 questions including at least one coding).
- **Thu:** Review mistakes from mock, revisit tricky topics (e.g. recursion base cases, off-by-ones).
- **Fri:** Study behavioral answers again, maybe do a practice behavioral interview with a friend.
- **Sat:** Light day – solve a puzzle or read Amazon news/culture (to show company interest).
- **Sun:** Rest and relaxation – ensure you're mentally fresh.

Week 4: Final Review & Practice.

- **Mon:** Review Amazon interview tips and LPs. Re-take any official Amazon OA practice test (Amazon provides an OA practice link) [1].
- **Tue:** Quick run-through of flashcards: common algorithms, Big-O of common operations, and quick patterns (two pointers, sliding windows).
- **Wed:** Final mock interview (with full 45–60 min tech+behavior).
- **Thu:** Review any final questions, get a good night’s sleep.

- **Fri:** Interview day! Be well-rested, arrive early (or set up tech), and have confidence.

Resources: Use the Amazon SDE OA Prep Course [1], LeetCode and GeeksforGeeks "Amazon" problem lists, HackerRank challenges, and behavioral prep guides. Consider scheduling 1-2 mock interviews per week on platforms like Pramp or interviewing.io.

Topic Frequencies & Practice Problems

Topic	**Frequency**	**Examples / Practice** (Problems & Sources)
Arrays (incl. Strings)	**Very high** (common) [4] [2]	*Two Sum* LeetCode (Easy); *Best Time to Buy/Sell* LC (Easy); *Reorganize String* LC (Med) [31]; *Min Window Substring*, *Subarray Sum*, *Kadane's*.
Strings	**High** (OA focus) [2]	*Longest Substring No Repeat* [LC]; *Valid Palindrome*; *Remove Adjacent Dups* [19]; *String Shuffle* (GfG).
Linked Lists	**Medium**	*Reverse Linked List*; *Cycle Detection (Floyd)*; *Merge K Lists*; *Copy List with Random Ptr*. Practice on [GfG Lists][209][210].
Trees/Tries	**Medium**	*Invert Binary Tree*; *BFS Level Order*; *LCA*; *Serialize/Deserialize Tree*; Trie insert/search; *Word Dictionary (Trie)*.
Graphs	**Medium**	*Course Schedule* [LC 207] (cycle check); *Pacific Atlantic* [LC 417] [8]; *Number of Islands*; *Graph Paths*; *Topological Sort* [4].
Dynamic Programming	**Medium**	*Coin Change*; *Knapsack (0/1)*; *LCS*; *Partition Equal Subset*; *Jump Game*; *Stock II*. Use GeeksforGeeks DP section.
Bit Manipulation	**Low–Medium**	*Single Number*; *Count 1 Bits*; *Power of Two/Three*; *XOR puzzles*. [GfG Bit Tricks][244].
Math / Complexity	**Occasional**	*Prime check*; *GCD/LCM*; *Modular exponentiation*; Big-O analysis questions.
Coding Puzzles	**Rare**	*Sudoku Solver*; *N-Queens*; logic puzzles. (Test problem-solving.)
Behavioral / LP	**Very high**	Prepare answers for common LPs: *Ownership*, *Customer Obsession*, *Bias for Action*, *Disagree & Commit*, *Learn & Be Curious*, etc. [5] [6].

Table: Topic areas seen in Amazon intern interviews with frequency and sample problems. For example, array/string problems (like Two Sum, Best Time, Reorganize String) are very common [31] [2], while advanced puzzles or design questions are rarer. The "Amazon SDE Sheet" prep list concurs: key topics include arrays (matrix), binary trees, BSTs, and linked lists, with later rounds emphasizing trees and graphs [4].

Sources: Amazon's official interview guidelines [1] [5], recent intern experiences [7] [8] [9], and aggregated prep guides [4]. Tables link to primary problem sources (LeetCode, GfG). Each code snippet and strategy above is drawn from these references and standard algorithms.

1 Online assessment prep for SDE roles
https://amazon.jobs/content/en/how-we-hire/university/sde-oa

2 Amazon SDE Intern Interview - Discuss - LeetCode
https://leetcode.com/discuss/post/6428663/Amazon-SDE-Intern-Interview/

3 Amazon SDE Interview Experience (On Campus) | by Freeze Francis | Jul, 2021 | Medium | InterviewNoodle
https://interviewnoodle.com/amazon-sde-interview-experience-on-campus-e8444ee791b?gi=35122b8a083c

4 Amazon SDE Sheet: Interview Questions and Answers - GeeksforGeeks
https://www.geeksforgeeks.org/dsa/amazon-sde-sheet-interview-questions-and-answers/

5 6 22 23 24 25 Leadership Principles
https://www.amazon.jobs/content/en/our-workplace/leadership-principles

7 26 30 Amazon | Internship SDE interviews | All stages | All Questions - Discuss - LeetCode
https://leetcode.com/discuss/post/1826560/amazon-internship-sde-interviews-all-stages-all-questions/

8 12 21 28 Amazon SDE Intern(6m) Interview Experience - Discuss - LeetCode
https://leetcode.com/discuss/post/7320620/

9 14 16 17 18 27 Amazon SDE Intern Interview Experience 6 Months - Discuss - LeetCode
https://leetcode.com/discuss/post/7332893/

10 11 15 20 Amazon SDE Internship (6 months) Interview Experience — On Campus | by Taanyatarun | Medium
https://medium.com/@taanyatarun/amazon-sde-internship-6-months-interview-experience-on-campus-6ca54210a894

13 Amazon SDE Intern Interview Experience - San Francisco, California
https://www.jointaro.com/interviews/companies/amazon/experiences/sde-intern-san-francisco-california-november-12-2025-accepted-offer-positive-42e84b55/

19 Python Program To Recursively Remove All Adjacent Duplicates - GeeksforGeeks
https://www.geeksforgeeks.org/dsa/python-program-to-recursively-remove-all-adjacent-duplicates/

29 31 Amazon SDE Intern Interview Experience - United States
https://www.jointaro.com/interviews/companies/amazon/experiences/sde-intern-united-states-october-30-2025-accepted-offer-positive-7304cbdb/